



FUNDAMENTOS DEL TESTING.

MATERIA: DISEÑO SEGÚN PLATAFORMA DE JUEGOS
ING. DANIELA RAMIREZ

¿QUE ES LA **CALIDAD**?

- La **calidad** de un videojuego es principalmente la ausencia de bugs
- Tenemos que **gestionar las expectativas de los jugadores**
- Un bug es un comportamiento que se sale de las expectativas del jugadores
- Un videojuego que no tiene bugs puede ser extremadamente aburrido



¿QUE ES EL TESTING Y POR QUE ES NECESARIO?

- **Las pruebas de software son un proceso, o una serie de procesos, diseñados para garantizar que el código informático, hace lo que debe hacer y, a la inversa, que no hace nada no previsto. El software debe ser predecible y coherente, sin sorpresas para los usuarios**

2011, p. 2)

(J. Myers,

System Error

**League of Legends Critical
Error Fix**



A critical error has occurred and the process must be terminated.

Would you like to create a crash dump to aid the developers in troubleshooting this issue? This may take up to 5 minutes.

NOTE: The process may appear unresponsive during this time.

Yes

No



¿QUE ES EL TESTING Y PORQUE ES NECESARIO?

- El testing de Software es un conjunto de actividades para descubrir defectos y evaluar la calidad de los artefactos del software (ISTBQ, 2023, p.16)
- Un testing Software que no funciona correctamente puede acarrear muchos problemas, como perdidas económicas, de tiempo o de reputación empresarial y, en casos extremos, incluso lesiones o la muerte. El testing software evalúa la calidad del software y ayuda a reducir el riesgo de fallo del software en operación.

BREAKING

New York Stock Exchange's 'Manual Error' Briefly Wiped Billions Of Dollars In Market Value—Here's What Happened

Jonathan Ponciano Forbes Staff

Follow

Jan 25, 2023, 11:14am EST

EMERGING TECH INFRASTRUCTURE MANAGED SERVICES & OUTSOURCING

Could software quality assurance processes have prevented the Boeing 737 MAX crash?

LYNN GREINER

MAY 6, 2019



Facebook



Twitter



LinkedIn



Reddit



Email



Print



Listen with Speechify

0:00

6:16

Related Tech News



World Backup Day: Time to think about the task few IT pros...

Howard Solomon - March 31, 2023



HP's next generation of printers aimed at sustainability and hybrid work unveiled...

Jim Love - March 30, 2023



Budget 2023 environmental component earns solid thumbs up from assorted organizations

Paul Barker - March 29, 2023



Government of Canada invests \$36 million in Ottawa semiconductor firm Ranovus and...

Ashee Pamma - March 29, 2023



TD Bank announces immersive VR experiences for colleagues and customers

Samira Balsara - March 28, 2023

Subscribe to our Newsletter

Our experienced team of journalists and bloggers bring you engaging in-depth

OBJETIVOS DEL TESTING

- Evaluar productos de trabajo como requisitos, historias de usuarios, diseños y código.
- Provocar fallos y encontrar defectos.
- Asegurar la cobertura necesaria de un objeto de prueba.
- Reducir el nivel de riesgo asociado a una calidad inadecuada del software.
- Verificar si se han cumplido los requisitos especificados.
- Verificar que un objeto de prueba cumple los requisitos contractuales, legales y normativos.
- Proporcionar información a los implicados para que puedan tomar decisiones informadas.
- Generar confianza en la calidad del objeto de prueba.
- Validar si el objeto de prueba está completo y funciona como esperaban los implicados

ERRORES, DEFECTOS, FALLOS Y CAUSAS

ERROR

Lo que hace un ser humano al equivocarse. Por ejemplo, piensa mal la lógica del código que está programando.

Produce un (varios) defecto(s).

DEFECTO

Algo incorrecto creado por un ser humano.

Puede producir un fallo.

FALLO

Si se ejecuta un código con un defecto, producirá fallo. También puede ocurrir debido a otros motivos: CPU defectuosa o el sobrecalentamiento de un componente.

CAUSA RAÍZ

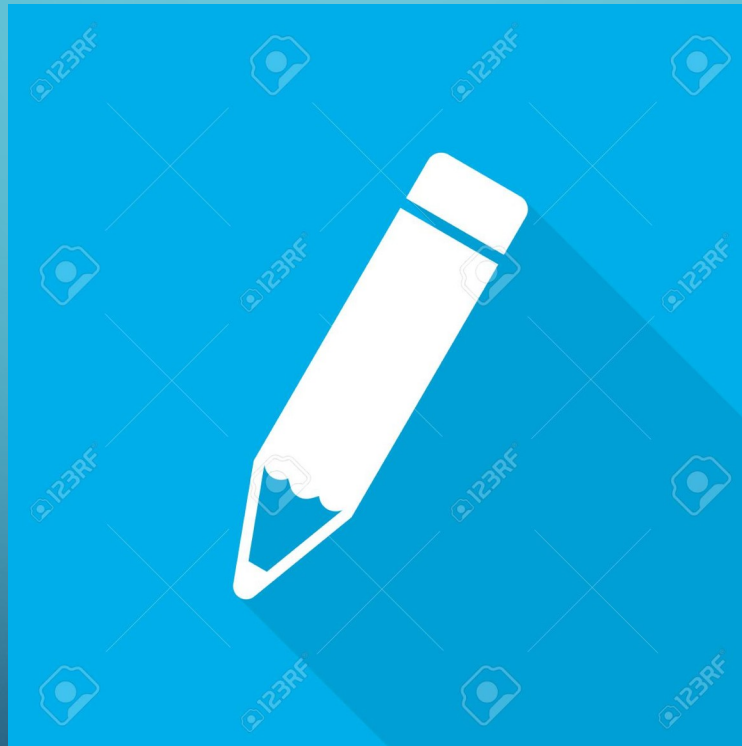
La causa original por la que se produce un error.

ERRORES, DEFECTOS, FALLOS Y CAUSAS

Ejemplo:

1. Un programador sufre una ruptura amorosa.
2. Al estar desconcentrado, tiene un error al pensar la lógica del código que está programando.
3. Esto le lleva a crear el siguiente defecto: en vez de escribir " ≤ 0 " en una comparación, ha escrito " < 0 ".
4. Cuando el código se ejecuta y la variable toma el valor "0", se produce un fallo (el comportamiento real no coincide con el comportamiento esperado).

¿ES ESTO ALGO SENCILLO DE TESTEAR?



TEST CASES DE LÁPIZ

Funcionalidad básica: debería escribir.

Condiciones límite: la mina debería estar presente dentro de la madera.

Condiciones de estrés: el lápiz no se rompe al ser sujetado o caerse al suelo.

Usabilidad: el lápiz es fácil de ser agarrado.

Security: agarrar el lápiz no es perjudicial para la salud.

Accesibilidad: ¿puede la gente daltónica leer el texto escrito en lápiz?

Testing de edad: ¿puedo seguir usando el lápiz en 2030?

Experiencia de usuario: ¿puedo coger el lápiz cómodamente?

Eficiencia: ¿cuánto puedo escribir con el lápiz hasta que la mina se termine?

Interfaz de usuario: ¿tiene el lápiz las dimensiones esperadas?

TIPOS DE TESTING

Reproducción de fallos

Método científico para reproducción de fallos:

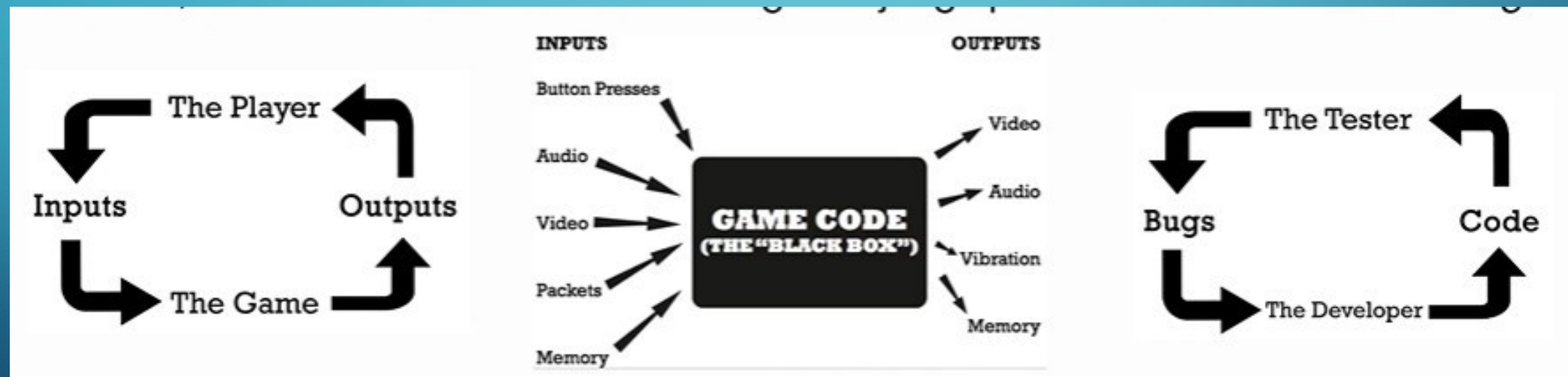
1. Observa un fenómeno. Por ejemplo, un fallo.
2. Desarrolla una hipótesis de qué creó el fenómeno.
3. Utiliza la hipótesis para hacer una predicción. Por ejemplo: si pierdo el nivel 3.952, el juego congela.
4. Testea tu hipótesis aplicándolo en el producto.
5. Repítelo varias veces para comprobar su reproducibilidad.



Técnicas de Caja Negra

Las técnicas de cajas negras se basan en un análisis del comportamiento especificado del objeto de prueba sin hacer referencia a su estructura interna (ISTQB, 2023, p.46).

Es decir, no necesitamos conocer el código del juego para llevar a cabo este testing.



Técnicas de Caja Blanca

Las técnicas de caja blanca se basan en la estructura interna y el procesamiento del objeto de prueba (ISTQB, 2023, p.46).

Es decir, necesitamos conocer el código del juego para llevar a cabo este testing. Permiten al tester probar el código fuente de maneras que el jugador real nunca podría hacer (Schultz, Bryant, 2016, p. 174).

Resulta muy útil para automatizar. Por ejemplo, podemos comprobar que las funciones del código devuelven lo que esperamos. Además, es muy útil poder referenciar elementos internos del funcionamiento del juego (como GameObjects en Unity) para poder verificar que funcionan bien, o referenciar variables.

Testing Funcional

Evalúa las funciones que debe realizar un componente o sistema. Las funciones son "aquello" que debe hacer el objeto de prueba (ISTQB, 2023, p.33).

Por ejemplo

Pulsar sobre el botón de guardar, guarda la partida.

Pulsar el botón de abrir el menú, abre el menú.

En Minecraft, si recogemos 3 manzanas de un árbol, nos sumará en 3 unidades la cantidad de manzanas que llevamos en el inventario



Testing No Funcional

Evalúa atributos distintos de las características funcionales de un componente o sistema. Es la comprobación de "lo bien que se comporta el sistema. La clasificación es la siguiente:

(ISTQB, 2023, p.33), (estándar ISO/IEC 25010):

- Eficiencia de desempeño
- Compatibilidad
- Usabilidad
- Fiabilidad
- Seguridad
- Mantenibilidad
- Portabilidad

